

ISTE STUDENTS CHAPTER (KA- 109)

21st Karnataka State Level ISTE Student Convention, 2025 *“Next-Gen Engineering: Innovating for India’s Deep Tech Future”*

Date: 11th and 12th November 2025

Venue: Yeneploya Institute of Technology, Moodbidri

PROBLEM STATEMENTS – AIML DOMAIN

1. Low Latency Audio RAG System -PACE Wisdom

Overview

Design and implement a real-time audio-based Retrieval-Augmented Generation (RAG) system that processes voice input and delivers audio responses with minimal latency. The system must find relevant knowledge sources for every query and provide accurate, contextual responses through a seamless audio interface.

Problem Description

Traditional RAG systems primarily work with text-based inputs and outputs, limiting their accessibility and user experience. There is a growing need for conversational AI systems that can process natural speech, retrieve relevant information from knowledge bases, and respond with synthesized speech in real-time. The challenge lies in creating a system that maintains accuracy while achieving sub-second response times.

System Requirements

Core Functionality

- **Audio Input Processing:** Accept voice input from users through microphone or audio streams
- **Real-time Speech Recognition:** Convert speech to text with high accuracy
- **Knowledge Retrieval:** Query vector database to find relevant documents for every user question
- **Response Generation:** Generate contextual responses using retrieved information

ISTE STUDENTS CHAPTER (KA- 109)

- **Audio Output:** Convert generated text responses back to natural-sounding speech
- **Live Streaming:** Stream audio output directly to the user interface as it's generated

Technical Specifications

Latency Requirements

- **Target Response Time:** Sub-2 second end-to-end latency
- **Audio Streaming:** Time-to-first-audio (TTFA) under 1 second
- **Real-time Processing:** Continuous audio input/output without interruption

Audio Quality Standards

- **Input:** Support for standard audio formats (WAV, MP3, WebM)
- **Output:** High-quality synthesized speech with natural intonation
- **Sample Rate:** Minimum 16kHz, preferably 22kHz or higher
- **Noise Handling:** Robust performance in moderate background noise

Accuracy Requirements

- **Speech Recognition:** >95% accuracy for clear speech
- **Retrieval Precision:** Relevant documents retrieved for 90%+ of queries
- **Response Quality:** Factually accurate and contextually appropriate responses

Technical Architecture

System Components

1. Audio Input Module

- **WebRTC Integration:** Real-time audio capture from browser/device
- **Audio Preprocessing:** Noise reduction and format standardization
- **Streaming Buffer:** Continuous audio stream management

2. Speech-to-Text (STT) Service

- **Recommended Models:**
 - OpenAI Whisper API for accuracy
 - Sarvam AI for Indian language support

ISTE STUDENTS CHAPTER (KA- 109)

- Real-time streaming STT services
 - **Features:** Punctuation, speaker diarization (optional)
 - **Languages:** English (primary), with extensibility for other languages
3. RAG Pipeline
- **Vector Database:** ChromaDB (local) or Zilliz Milvus (cloud free tier)
 - **Embedding Model:** Sentence transformers or OpenAI embeddings
 - **Document Processing:** Text chunking, metadata extraction
 - **Retrieval Strategy:** Semantic similarity search with configurable k-values
4. Language Model Integration
- **Framework:** LangChain or LlamaIndex for RAG orchestration
 - **LLM Options:** OpenAI GPT-3.5/4, Anthropic Claude, or open-source alternatives
 - **Prompt Engineering:** Optimized prompts for accuracy and conciseness
5. Text-to-Speech (TTS) Service
- **Recommended Services:**
 - ElevenLabs for high-quality, low-latency streaming
 - OpenAI TTS for reliability
 - Sarvam AI for Indian language support
 - **Streaming Capability:** Real-time audio generation and streaming
 - **Voice Selection:** Multiple voice options with consistent quality
6. User Interface
- **Frontend Framework:** Streamlit for rapid prototyping
 - **WebSocket Communication:** Real-time audio streaming
 - **Visual Components:** Audio waveform display, conversation history, system status
 - **Responsive Design:** Works across desktop and mobile browsers

Implementation Guidelines

Development Stack

- **Backend:** Python 3.8+
- **Web Framework:** Streamlit with streamlit-webrtc component
- **RAG Framework:** LangChain or LlamaIndex
- **Vector Database:** ChromaDB (development) / Zilliz Milvus (production)
- **Audio Processing:** WebRTC, PyAudio for local processing
- **Real-time Communication:** WebSockets for audio streaming

ISTE STUDENTS CHAPTER (KA- 109)

Database Setup

ChromaDB (Local Development)

Zilliz Milvus (Cloud Production)

- **Free Tier:** 5GB storage, 2.5M vCUs monthly, up to 5 collections
- **Setup:** Create cluster on Zilliz Cloud, obtain connection credentials
- **Integration:** Use pymilvus client for connection and operations

Audio Pipeline Architecture

Streaming Workflow

1. **Audio Capture:** Continuous microphone input via WebRTC
2. **Real-time STT:** Stream audio chunks to speech recognition service
3. **Query Processing:** Trigger RAG pipeline on sentence completion
4. **Parallel Processing:** Overlap retrieval, generation, and TTS synthesis
5. **Audio Streaming:** Stream synthesized audio back to client

Optimization Strategies

- **Caching:** Cache frequent queries and responses
- **Preprocessing:** Audio noise reduction and normalization
- **Batching:** Efficient batch processing for embeddings
- **Connection Pooling:** Reuse connections to external APIs

Data Requirements

Knowledge Base

- **Document Types:** Text files, PDFs, web pages, structured data
- **Volume:** Minimum 50 to 100 documents for meaningful testing
- **Processing:** Automatic chunking, embedding generation, metadata extraction
- **Updates:** Support for real-time knowledge base updates

Sample Data Sources

- FAQ documents
- Technical documentation
- Educational content

ISTE STUDENTS CHAPTER (KA- 109)

- Domain-specific knowledge bases

Evaluation Criteria

Primary Metrics

Latency Measurements

- **End-to-end Response Time:** From voice input completion to audio output start
- **Component Latency:** Individual timing for STT, retrieval, generation, TTS
- **Streaming Performance:** Time-to-first-audio and continuous playback quality

Accuracy Assessment

- **STT Accuracy:** Word Error Rate (WER) on test audio samples
- **Retrieval Quality:** Precision@k, recall@k, NDCG metrics
- **Response Quality:** Human evaluation for factual accuracy and relevance
- **Citation Accuracy:** Proper attribution to source documents

System Performance

- **Throughput:** Concurrent user support capability
- **Resource Usage:** CPU, memory, network bandwidth consumption
- **Error Handling:** Graceful degradation and recovery mechanisms

Secondary Metrics

User Experience

- **Audio Quality:** Naturalness and clarity of synthesized speech
- **Conversation Flow:** Smooth interaction without interruptions
- **Response Completeness:** Comprehensive answers with proper context

Technical Robustness

- **Error Rate:** Frequency of system failures or incorrect responses
- **Scalability:** Performance under increasing load
- **Reliability:** Consistent performance across different audio conditions

ISTE STUDENTS CHAPTER (KA- 109)

Deliverables

Core Application

1. **Functional Audio RAG System:** Complete working implementation
2. **Web Interface:** User-friendly Streamlit application
3. **Documentation:** Setup guide, API documentation, user manual
4. **Configuration Files:** Environment setup, model configurations

Technical Artifacts

1. **Source Code:** Well-commented, modular Python implementation
2. **Database Schema:** Vector database setup and indexing strategy
3. **Test Suite:** Unit tests, integration tests, performance benchmarks
4. **Deployment Guide:** Instructions for local and cloud deployment

Evaluation Report

1. **Performance Analysis:** Comprehensive latency and accuracy measurements
2. **Comparison Study:** Benchmarking against existing solutions
3. **Optimization Recommendations:** Identified improvements and next steps
4. **Demo Video:** System demonstration with real-world use cases

Success Criteria

Minimum Viable Product (MVP)

- ☐ Sub-2 second end-to-end response time for 80% of queries
- ☐ >90% speech recognition accuracy on clear audio
- ☐ Successful retrieval of relevant documents for 85% of queries
- ☐ Natural-sounding audio output with minimal artifacts
- ☐ Stable WebRTC connection with continuous audio streaming

Advanced Features (Bonus)

- ☐ Multi-language support (English + 1 Indian language)
- ☐ Voice activity detection for hands-free operation
- ☐ Conversation memory and context awareness
- ☐ Real-time knowledge base updates
- ☐ Mobile-responsive interface

ISTE STUDENTS CHAPTER (KA- 109)

Performance Benchmarks

- ☐ Handle 10+ concurrent audio streams
- ☐ Process 1000+ documents in knowledge base
- ☐ Maintain <5% error rate across all components
- ☐ Support 30+ minute continuous conversations
- ☐ Deploy successfully on cloud infrastructure

Getting Started

Prerequisites

- Python 3.8 or higher
- Node.js (for WebRTC components)
- API keys for chosen STT/TTS services
- Vector database setup (local or cloud)

Quick Start Steps

1. Environment Setup: Install required Python packages
2. API Configuration: Set up credentials for external services
3. Database Initialization: Create and populate vector database
4. Service Integration: Configure STT, LLM, and TTS services
5. Interface Development: Build Streamlit application with WebRTC
6. Testing: Validate end-to-end functionality
7. Optimization: Fine-tune for latency and accuracy requirements

Resources and References

Documentation Links

- [GitHub - whitphx/streamlit-webrtc: Real-time video and audio processing on Streamlit](#)
- [Build a RAG agent with LangChain - Docs by LangChain](#)
- [Getting Started - Chroma Docs](#)
- [Streaming | ElevenLabs Documentation](#)
- [Create AI Applications with Zilliz Cloud for Free](#)

Technical Papers

- “Low-Latency End-to-End Voice Agents” (2024)
- “Streaming RAG Approach to Real-time Knowledge Base” (2025)
- “RAG Evaluation: Metrics and Benchmarks” (2025)

ISTE STUDENTS CHAPTER (KA- 109)

This problem statement provides a comprehensive framework for developing a production-ready, low-latency audio RAG system with clear technical requirements, evaluation criteria, and implementation guidelines.

2. AI for Local Language Learning -Microsoft

Problem Statement:

Develop an AI-powered platform to help students and migrants learn and practice the regional local language (e.g., Kannada, Tamil, Telugu, Marathi, Hindi). The system should cover all **LSRW (Listening, Speaking, Reading, and Writing)** modules using natural language processing, voice recognition, and gamification to make learning engaging and adaptive.

Expected Outcome:

- AI-based pronunciation and grammar feedback (Speaking & Writing modules)
- Conversational chatbot for real-life language practice (Listening & Speaking modules)
- Multi-level gamified lessons and progress tracking (All LSRW modules)
- Speech and text translation modules for inclusivity (Listening & Reading modules)

Impact:

Bridges linguistic gaps, supports cultural integration, and strengthens digital education in regional languages through comprehensive **LSRW-based language learning**.

3. AI-Powered Personalized Learning Companion

Problem Statement:

Develop an autonomous AI agent that enhances the learning experience by providing personalized educational support, real-time feedback, and adaptive content generation across the subjects: Research Methodology

ISTE STUDENTS CHAPTER (KA- 109)

and Intellectual Property Rights, Data Structures and Applications, Discrete Mathematical Structures. The agent should intelligently understand and respond to student queries, track individual progress, and tailor study materials such as quizzes, summaries, and explanations to meet unique learning needs. The aim is to create a versatile, user-focused educational assistant that improves knowledge retention and engagement through AI-driven interaction and dynamic learning pathways.

4. AI-Powered Bowling Action Validation

The Challenge:

Cricket bowlers, particularly fast bowlers, are susceptible to career-threatening injuries and the risk of illegal bowling actions (chucking) if their technique deviates from established biomechanical standards (e.g., the 15-degree elbow extension limit set by the ICC). Currently, sophisticated, high-cost biomechanical labs or subjective on-field umpires are the primary methods for action analysis and validation. This is inaccessible to local clubs, academies, and amateur players, where early detection of flaws could prevent injuries and enforce fair play.

Objective:

Design and develop a cost-effective, accessible, and automated AI/Machine Learning solution that can analyze and validate a bowler's action in real-time or from standard video footage.

Specific Requirements (Focus Areas for Teams)

Teams must focus on developing an application or system that can perform the following tasks with high accuracy:

- **Kinematic Analysis:** Accurately track and analyze key joint angles and movement parameters during the bowling sequence (e.g., point of release, front-foot contact, and the most critical phase: elbow extension).
- **Angle Measurement & Validation:** Calculate the maximum elbow extension angle from the point of arm straightening to the

ISTE STUDENTS CHAPTER (KA- 109)

point of ball release. The system must flag an action as "illegal" if this angle exceeds the established 15-degree threshold.

- **Visualization & Feedback:** Provide instantaneous, clear, and actionable feedback to the bowler and coach, including a visual overlay of tracked joint paths and the calculated elbow extension angle.
- **Accessibility:** The solution should ideally work using off-the-shelf camera equipment (e.g., a standard smartphone camera or webcam) rather than specialized, high-speed cameras.

5. Intelligent Traffic Prediction and Route Optimization System

Objective:

Develop an AI-powered system that predicts traffic congestion, analyzes traffic patterns from video data, recommends optimal routes, and provides insights for better urban mobility. The solution should demonstrate practical applications of machine learning in solving real-world traffic challenges using available datasets and video samples.

Core Requirements:

Build traffic flow prediction models that forecast congestion levels for the next 30 minutes to 3 hours at major road segments using historical traffic data, achieving minimum 70% prediction accuracy. Implement computer vision capabilities to detect and count vehicles from traffic video footage, classify vehicle types (cars, buses, trucks, two-wheelers), and estimate traffic density. Create a route recommendation engine that suggests optimal paths between two points considering current traffic conditions, predicted congestion, and provides 2-3 alternative routes with estimated travel times. Develop an interactive dashboard visualizing real-time traffic conditions with color-coded heat maps, displaying prediction results with confidence scores, showing historical traffic patterns and trends, and comparing multiple routes side-by-side.

Advanced Features:

Implement incident detection identifying unusual traffic patterns, what-if scenario simulator for testing traffic interventions, public transport

ISTE STUDENTS CHAPTER (KA- 109)

integration analysis, and automated report generation for traffic authorities.

Deliverables:

Working prediction models, vehicle detection system processing sample videos, route optimization engine, interactive web dashboard, documentation, demonstration video, and presentation explaining approach and results.